Artificial Neural Networks

Assignment #3

1. What is a scoring function and a loss function? Explain each with a suitable example.

a. Scoring Function: Scoring Function is a mathematical formula that takes the input data and produces scores for the possible classes where the highest score might become the predicted career

Formula: For Linear Classification the formula is

$$f(x) = wx + b$$

Where:

- w: weight
- x: Input value
- b: bias

b. Loss Function: Loss Function is a mathematical formula that measures how good or bad the model's prediction was based on the difference between actual value vs predicted value.

Formula (Mean Squared Error):

$$L = (y - f(x))^2$$

Where:

- L = Calculated loss value
- y = Actual true value
- f(x) = Predicted score

2. Differentiate between simple loss, hinge loss, and cross-entropy loss.

a. Simple Loss: Simple loss (0-1 loss) is a loss function that assigns a loss of 0 when the predicted label matches the true label, and a loss of 1 when it does not. It is also called 0-1 Loss.

Formula:

$$L = 0, \text{ if } \hat{y} = y$$
$$L = 1, \text{ if } \hat{y} != y$$

Where:

- L = Calculated loss value
- y = Actual true value
- $\hat{y}$ = Predicted score

b. Hinge Loss: Hinge loss, also known as multi-class SVM loss, is a loss function that checks whether the score of the correct class is higher than the scores of the wrong classes by at least 1. If it is, the loss is 0. If not, the loss gets bigger the more it misses.

Formula:

$$L = \Sigma \max(0, s_j - s_\gamma + 1) \quad \text{for all } j \mathrel{!}= y$$

Where:

- L = Calculated loss value
- s_j = Score of an incorrect class
- s_γ = Score of the correct class
- 1 = Margin

c. Cross Entropy Loss: Cross-entropy loss is a loss function that is used with the softmax classifier. It converts the scores into probabilities and punishes the model when the correct class gets low probability. When the correct class gets high probability, the loss is small.

Formula:

$$L = -\ln(e^{(s_\gamma)} / \Sigma e^{(s_j)})$$

Where:

- L = Calculated loss value
- s_γ = Score of the correct class
- $\Sigma e^{(s_j)}$ = Sum of e^(score) of all classes

3. Gradient Descent

a. Types of Gradient Descent: Gradient descent is an optimization method that is used to find the weights W and bias b that give the minimum loss. In every step, it calculates the gradient of the loss and moves the weights a small step in the opposite direction. There are three types of gradient descent:

- **Batch Gradient Descent (Vanilla):** It uses the whole training set to calculate one gradient and makes only one update per epoch. It is smooth and stable, but it is slow when the dataset is large.
- **Stochastic Gradient Descent (SGD):** It uses only one training example to calculate the gradient and updates the weights after every example. It is fast, but the updates are noisy.
- **Mini-Batch Gradient Descent:** It uses a small group of examples (like 32, 64, 128 or 256) for each update. It is a balance between the other two types and it is the most used type in deep learning.

Formula:

$$W = W - \alpha * \text{gradient}$$

Where:

- W: Weights (and bias)
- α: Learning rate (size of the step)

- gradient: Derivative of the loss with respect to W

b. Batch Gradient Descent: In batch gradient descent, all the training data is used to calculate the gradient, and the weights are updated once per epoch. The example used here is predicting the exam score of a student from the study hours (100 students). The model is score = w * hours + b.

Code:

```python
import numpy as np
# Data: study hours vs exam score (score = 35 + 6 * hours + noise)
rng = np.random.default_rng(42)
hours = rng.uniform(0, 10, 100)

score = 35 + 6 * hours + rng.normal(0, 4, 100)
w, b = 0.0, 0.0
alpha, epochs = 0.01, 100

for epoch in range(epochs):

    error = (w * hours + b) - score
    w -= alpha * np.mean(error * hours)    # gradient for w
    b -= alpha * np.mean(error)            # gradient for b
    print(f"Error: {error}")

loss = np.mean(((w * hours + b) - score) ** 2) / 2
print(f"w = {w:.3f}, b = {b:.3f}, loss = {loss:.3f}")
```

Result:

```
   0.47632091  -1.24926364  -5.1597122   -1.08348367   1.3775027
  -4.66323733  -1.6890145   -6.22580672  -0.94127257   4.83145225
   5.26055984  -6.58857602  -6.98401171   0.60152346   1.38298779
  -5.85822306   4.30360659   3.33982405  -2.7997126    1.70447827]
Error: [ -1.55157169   3.5295782    1.59550542  -5.18135623   1.18883774
  -2.81836043   3.77664632   0.8743295    3.57803526   1.26859758
  -3.48198771   7.0205902   -1.74447003  -0.88304548   2.28615505
   5.60355382  -0.33255958   1.86908022  -0.86077534  -0.09927668
  -6.36621387   0.82987531   4.22363637  -0.61983717  -0.83061348
  -5.63103533  -3.42116136  -1.6847286   -6.06433857   4.76463231
   2.59434566   3.83461935   1.41971222   5.38584822  -2.6201369
   0.69244888   5.66192684   3.98534988  -1.43487047  -3.34845052
  -8.07998224 -11.5834137   -1.64087152   3.81300774   8.60004665
  -1.01046107   3.13797169   1.50625048   2.45773628   0.34596871
  -4.45600643  -0.90064951   0.68747762   4.14459912   6.71754339
   1.99558393   0.13115588  -7.10985224  -0.73827434  -4.15868085
   2.00064917   4.66088492   3.82087141   2.94469069  -8.52447231
   3.24108551  -3.39606896   3.46305086  -3.98895681  -1.63304392
   0.44059212   0.05804391   2.69983762  -1.96216947   1.56874151
   4.74434595   4.95876965  -0.68959237  -6.35938504  -0.58829132
   0.47631314  -1.24920474  -5.15975868  -1.08336288   1.37766078
  -4.66309666  -1.68903728  -6.22576217  -0.94115031   4.83148668
   5.26068441  -6.58859207  -6.98396309   0.60158892   1.3830738
  -5.85822204   4.30367701   3.33996534  -2.79957916   1.70439357]
w = 6.042, b = 34.725, loss = 7.671
(ai_env_tensorflow) waseem-dev@paxtos-fedora:~/AI/multi_modal_ai_engineering_journey/Artifi
cial Neural Networks/assignments/assignment_3/source_code$
```

The learned values w = 6.031 and b = 34.795 are very close to the real values 6 and 35 that were used to create the data.

c. Bias Trick: The bias trick is a method of combining the weight W and the bias b into one parameter. We add one extra column of 1s to the input data X, and b becomes the last value inside W.

Formula:

$$f(x) = Wx + b \quad \text{becomes} \quad f(x) = Wx$$

For example, with one feature, W = [w, b] and x = [hours, 1], so W · x = w * hours + b * 1.

Purpose: It is difficult to keep track of two separate variables W and b. With the bias trick, the bias becomes a trainable part of the weight matrix, so we have to store and update only one parameter, and the gradient is calculated with one formula.

Code:

```python
import numpy as np
# Data: study hours vs exam score (score = 35 + 6 * hours + noise)
rng = np.random.default_rng(42)
hours = rng.uniform(0, 10, 100)
score = 35 + 6 * hours + rng.normal(0, 4, 100)

X = np.c_[hours, np.ones(len(hours))]      # each row is [x, 1]

W = np.zeros(2)                            # W = [w, b]

alpha, epochs = 0.01, 2643

for epoch in range(epochs):

    error = X.dot(W) - score
    W -= alpha * X.T.dot(error) / len(score)
    print(error)

loss = np.mean((X.dot(W) - score) ** 2) / 2
print(f"w = {W[0]:.3f}, b = {W[1]:.3f}, loss = {loss:.3f}")
```

Result:

```
    4.74425474    4.9586816    -0.68958521   -6.35940499   -0.58825263
    0.47632091   -1.24926364   -5.1597122    -1.08348367    1.3775027
   -4.66323733   -1.6890145    -6.22580672   -0.94127257    4.83145225
    5.26055984   -6.58857602   -6.98401171    0.60152346    1.38298779
   -5.85822306    4.30360659    3.33982405   -2.7997126     1.70447827]
[  -1.55157169    3.5295782     1.59550542   -5.18135623    1.18883774
   -2.81836043    3.77664632    0.8743295     3.57803526    1.26859758
   -3.48198771    7.0205902    -1.74447003   -0.88304548    2.28615505
    5.60355382   -0.33255958    1.86908022   -0.86077534   -0.09927668
   -6.36621387    0.82987531    4.22363637   -0.61983717   -0.83061348
   -5.63103533   -3.42116136   -1.6847286    -6.06433857    4.76463231
    2.59434566    3.83461935    1.41971222    5.38584822   -2.6201369
    0.69244888    5.66192684    3.98534988   -1.43487047   -3.34845052
   -8.07998224  -11.5834137    -1.64087152    3.81300774    8.60004665
   -1.01046107    3.13797169    1.50625048    2.45773628    0.34596871
   -4.45600643   -0.90064951    0.68747762    4.14459912    6.71754339
    1.99558393    0.13115588   -7.10985224   -0.73827434   -4.15868085
    2.00064917    4.66088492    3.82087141    2.94469069   -8.52447231
    3.24108551   -3.39606896    3.46305086   -3.98895681   -1.63304392
    0.44059212    0.05804391    2.69983762   -1.96216947    1.56874151
    4.74434595    4.95876965   -0.68959237   -6.35938504   -0.58829132
    0.47631314   -1.24920474   -5.15975868   -1.08336288    1.37766078
   -4.66309666   -1.68903728   -6.22576217   -0.94115031    4.83148668
    5.26068441   -6.58859207   -6.98396309    0.60158892    1.3830738
   -5.85822204    4.30367701    3.33996534   -2.79957916    1.70439357]
w = 6.042, b = 34.725, loss = 7.671
(ai_env_tensorflow) waseem-dev@paxtos-fedora:~/AI/multi_modal_ai_enginee
```

The result is exactly the same as the result of part (b). This shows that the bias trick changes only the way of writing, not the answer.

d. Mini-Batch Gradient Descent: Mini-batch gradient descent divides the training data into small groups called mini-batches, and the weights are updated once for every mini-batch. The data is shuffled at the start of every epoch. Here the batch size is 20, so there are 100 / 20 = 5 mini-batches in every epoch.

Difference from Batch Gradient Descent:

- **Data per update:** Batch GD uses all 100 examples for one update, while mini-batch GD uses only 20 examples.
- **Updates per epoch:** Batch GD makes 1 update, while mini-batch GD makes 5 updates.
- **Speed:** Mini-batch GD learns faster per epoch because it updates the weights more often.
- **Loss curve:** Batch GD gives a smooth curve, while mini-batch GD is a little noisy.
- **Memory:** Mini-batch GD needs less memory because only one small batch is used at a time.

Code:

```python
import numpy as np
# Data: study hours vs exam score (score = 35 + 6 * hours + noise)

rng = np.random.default_rng(42)
hours = rng.uniform(0, 10, 100)
score = 35 + 6 * hours + rng.normal(0, 4, 100)

X = np.c_[hours, np.ones(len(hours))]     # bias trick
W = np.zeros(2)
alpha, epochs, batch_size = 0.01, 1000, 20

for epoch in range(epochs):

    idx = rng.permutation(len(score))     # shuffle every epoch

    X, score = X[idx], score[idx]

    for i in range(0, len(score), batch_size):

        bx, by = X[i:i + batch_size], score[i:i + batch_size]

        W -= alpha * bx.T.dot(bx.dot(W) - by) / len(by)    # update per batch


loss = np.mean((X.dot(W) - score) ** 2) / 2
print(f"w = {W[0]:.3f}, b = {W[1]:.3f}, loss = {loss:.3f}")
print(f"updates per epoch = {len(score) // batch_size}")
```

Result:

```
(ai_env_tensorflow) waseem-dev@paxtos-fedora:~/AI/multi_modal_ai_engineering_journey/Artificial Neural Networks/assignments/assignment_3/source_code$ python3 mini_batch.py
w = 6.079, b = 34.805, loss = 7.710
updates per epoch = 5
```

Loss after the same number of epochs: after epoch 1, batch GD has a loss of 1094.3 while mini-batch GD has 174.0. After epoch 100, batch GD has 89.25 while mini-batch GD has 20.30. Both methods reach almost the same line in the end, but mini-batch GD reaches it in far fewer epochs.

4. What are the main steps involved in the training of a Neural Network? Explain each step briefly.

Step 1. Prepare the data: Collect the data, scale it (for example pixel values between 0 and 1), convert the class labels into vectors, and split the data into training and testing sets.

Step 2. Initialize the weights and biases: All the weights are set to small random values at the start. The bias is added with the bias trick.

Step 3. Forward pass: The inputs are passed through the network layer by layer. At every layer we take the dot product of the inputs and the weights and then apply an activation function, until we get the final prediction.

Step 4. Calculate the loss: The prediction is compared with the true label using a loss function (such as hinge loss or cross-entropy loss). A smaller loss means a better prediction.

Step 5. Backward pass (backpropagation): The gradient of the loss is calculated at the output layer, and then the chain rule is used to send this gradient backward through the layers, so we know how much each weight affected the error.

Step 6. Update the weights: Each weight is moved a small step in the opposite direction of its gradient using gradient descent: W = W - α * gradient.

Step 7. Repeat: Steps 3 to 6 are repeated for many epochs until the loss becomes low and the accuracy is good. Finally, the network is evaluated on the testing data.

5. What is the role of the learning rate α in updating the parameters W and b? Explain the effect of using a learning rate that is too small or too large.

Role of the learning rate: The learning rate α controls the size of the step that we take in the direction opposite to the gradient when we update the parameters. The gradient tells us the direction, and α tells us how far to move.

Formula:

$$W = W - \alpha * (\text{gradient of } W)$$

$$b = b - \alpha * (\text{gradient of } b)$$

Too small α: The steps are tiny, so the learning is very slow and many epochs are needed. The training may also become stuck before reaching a good solution.

Too large α: The steps are too big, so the weights jump over the minimum. The loss goes up and down, and it can become bigger and bigger, which means the training fails (diverges).

Example (batch gradient descent on the study hours data, after 100 epochs):

- **α = 0.0001 (too small):** loss = 1226, w = 3.06 and b = 0.56 are still very far from the real values 6 and 35.
- **α = 0.01 (good):** loss = 89.25, and it keeps decreasing steadily.
- **α = 0.1 (too large):** the loss grows to a huge number (about 1.9 x 10^71), so the training fails.